

Requirement Analysis & Planning

1. Feature List & User Stories

Core Features

- User registration and login
- Product catalog with searching with filter and sort
- Shopping cart with features: add, update, remove items, view total
- Checkout cart for creating order
- Order history of user
- Set and calculate discount
- Admin role with for creating, editing, delete and listing products
- User and admin protected route via separation of duties principle

User Stories

Authentication

- As a visitor, I can register an account so I can make purchases.
- As a registered user, I can log in and stay logged in across page refreshes.
- As a user, I can log out.

Browsing & Discovery

- As a end user, I can view a list of all products with name, price, image, and category.
- As a end user, I can search products filter them and sort them by different categories.
- As a end user, I can view a single product's detail page.

Cart

- As a logged-in user, I can add a product to my cart.
- As a logged-in user, I can change the quantity of an item in my cart.
- As a logged-in user, I can remove an item from my cart.
- As a logged-in user, I can see my cart's subtotal, discount applied, and total.

Discounts

- As a login user, I can enter a coupon code at checkout to receive a discount.
- As a login user, I get a clear error if a code is invalid or expired.

Orders

- As a logged-in user, I can check out my cart to place an order.
- As a logged-in user, I can view a list of my past orders.
- As a user, my cart is cleared after a successful order.

Admin

- As an admin, I can create a new product.
- As an admin, I can edit an existing product.
- As an admin, I can delete a product.
- As an admin, I can view all products in a manageable list / table.
- As a non-admin, I cannot access any admin routes, front-end or back-end.

2. Data Model & API Endpoint List

Data Model

User Schema

Field	Type	Notes
name	String	required
email	String	required, unique
passwordHash	String	required, never returned in responses
role	String	enum: user, admin — defaults to user
createdAt / updatedAt	Date	automatic

Product Schema

Field	Type	Notes
name	String	required
description	String	required
price	Number	required, positive
category	String	required — used for filtering
imageUrl	String	optional
stock	Number	required, defaults to 0
createdAt / updatedAt	Date	automatic

Cart Schema *(One per user)*

Field	Type	Notes
user	ObjectId	required, references User, unique (one cart per user)
items	Array	[{ product: ObjectId (ref: Product), quantity: Number }]
createdAt / updatedAt	Date	automatic

Coupon Schema

Field	Type	Notes
code	String	required, unique, uppercase
discountPercent	Number	required, 1–100
expiresAt	Date	required

Field	Type	Notes
active	Boolean	defaults to true

Order Schema

Field	Type	Notes
user	ObjectId	required, references User
items	Array	[{ product, name, priceAtPurchase, quantity }] — snapshotted at order time
couponCode	String	optional
discountAmount	Number	defaults to 0
subtotal	Number	required
total	Number	required
status	String	enum: placed, completed — defaults to placed
createdAt / updatedAt	Date	automatic

API Endpoint List

Login/register

Method	Path	Auth	Description
POST	/api/auth/register	No	Register, returns JWT
POST	/api/auth/login	No	Login, returns JWT
GET	/api/auth/me	Yes	Get current user profile

Products

Method	Path	Auth	Description
GET	/api/products	No	List products — supports search, category, minPrice, maxPrice, sort
GET	/api/products/:id	No	Get one product
POST	/api/products	Admin only	Create a product
PUT	/api/products/:id	Admin only	Replace a product
PATCH	/api/products/:id	Admin only	Partially update a product
DELETE	/api/products/:id	Admin only	Delete a product

Cart

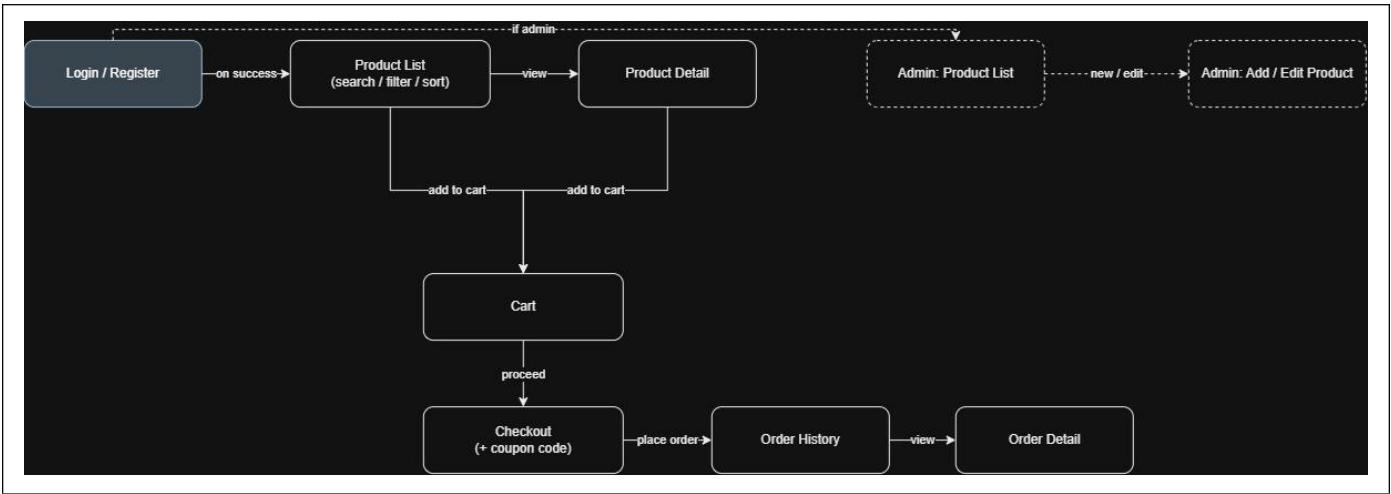
Method	Path	Auth	Description
GET	/api/cart	Yes	Get current user's cart
POST	/api/cart/items	Yes	Add an item to cart
PATCH	/api/cart/items/:productId	Yes	Update quantity
DELETE	/api/cart/items/:productId	Yes	Remove item from cart

Orders

Method	Path	Auth	Description
POST	/api/orders	Yes	Checkout — creates order from cart, clears cart
GET	/api/orders	Yes	List current user's past orders
GET	/api/orders/:id	Yes	Get one order's details

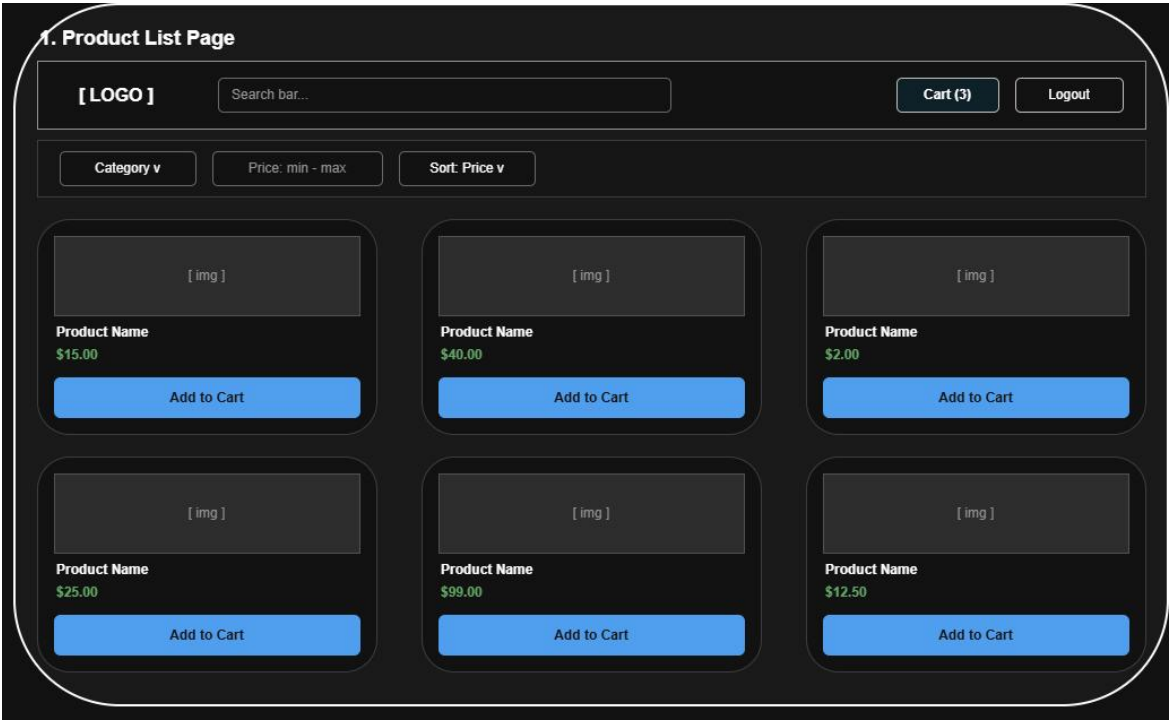
3. Wireframe / Component Sketch

Pages

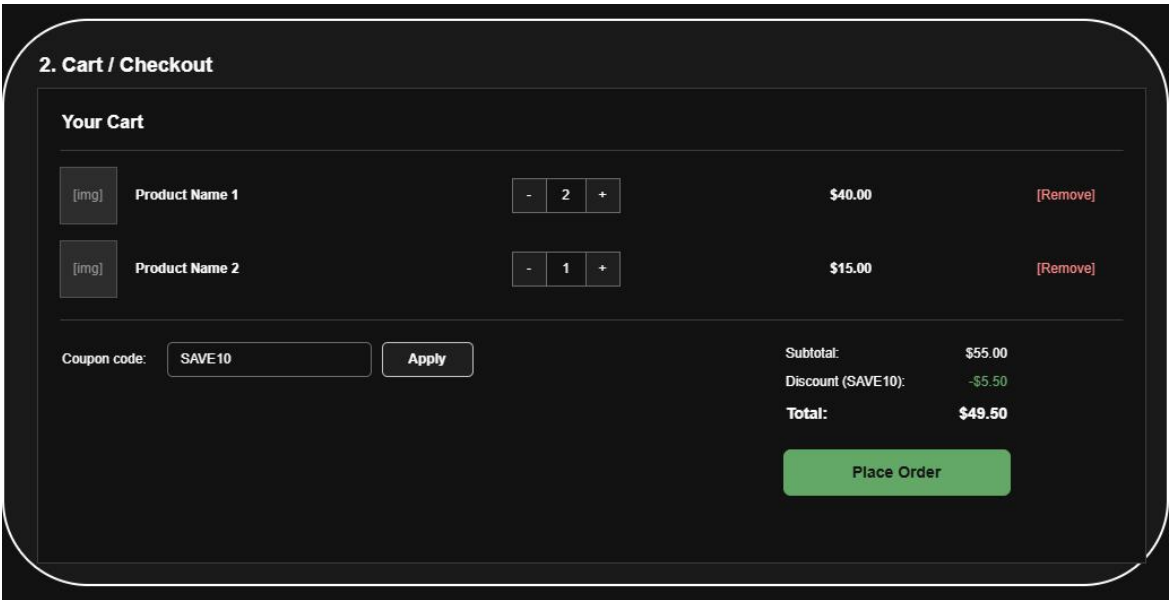


Rough Wireframe Sketch

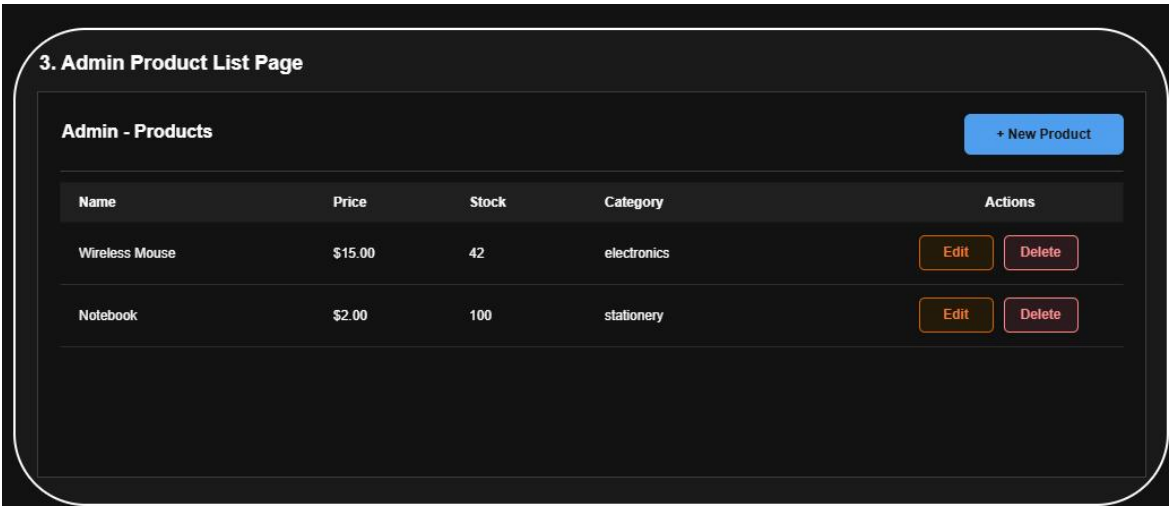
Product List Page



Cart / Checkout



Admin Product List



4. Repo + Task Board Setup

Repo Structure

```
mern-storefront/  
|-- client/           (React + Vite)  
|   |-- src/, package.json, .env.example  
|-- server/           (Node + Express)  
|   |-- routes/, controllers/, models/, middleware/, package.json, .env.example  
|-- .gitignore         (node_modules, .env)  
|-- README.md
```

Each feature branch merges into main via pull request once its slice is working, keeping main always in a demo-able state.